

Tema 2, 13 noiembrie 2020

Termen de predare: 11:00, 22 noiembrie 2020, prin e-mail

- Tema poate fi rezolvată în echipe de câte doi studenți.
- Pentru soluții (corect) redactate în LaTeX se oferă un bonus de 2 puncte.
- Nu rescrieți enunțurile! Nu este nevoie de mai mult de o pagină sau o pagină și jumătate pentru fiecare dintre probleme.
- ORICE SOLUȚIE COPIATĂ A UNEIA DINTRE PROBLEMELE DE MAI JOS VA FI PUNCTATĂ CU -2 PUNCTE.
- Soluțiile vor fi trimise prin e-mail până pe 22 noiembrie 2020 la ora 11:00 dimineața, la une dintre adresele: olariu@info.uaic.ro, fe.olariu@gmail.com
- E-mail-ul trebuie să conțină fișierul sursă (un fișier .doc, .odt sau .tex) și fișierul .pdf, amândouă cu următorul format al numelui:

Nume1_grupă_Nume2_grupă_Tema1.tex (sau .doc, .odt)

Nume1_grupă_Nume2_grupă_Tema1.pdf

- Exemple:

IonescuPVasile_A8_VasilescuTion_X1_Tema1.tex (sau .doc, .odt, .pdf)

IonescuPVasile_B6_VasilescuTion_E5_Tema1.tex (sau .doc, .odt, .pdf)

1. Fie $G = (S, T; E)$ un graf bipartit. G se numește σ -bipartit dacă $d_G(x) = d_G(y)$ pentru orice doua noduri x și y din aceeași clasă a bipartiției.

- (a) Arătați că într-un graf σ -bipartit există un cuplaj care saturează una din cele două clase ale bipartiției.
- (b) Doi prieteni interpretează următorul joc de cărți: primul dintre ei alege la întâmplare șase cărți dintr-un pachet (de 52) de cărți, se uită la ele, păstrează una dintre ele iar pe celelalte cinci le așează cu fața în sus ordonate de la stânga la dreapta; al doilea prieten ghicește corect cartea păstrată de primul. Arătați că cei doi prieteni se pot înțelege asupra unui sistem care să funcționeze întotdeauna și determinați un astfel de sistem.

(2 + 2 = 4 points)

2. Într-o rețea de computere există două tipuri de noduri: clienți și servere. Fiecare client este conectat direct (folosind cabluri) la cel puțin un server dar nu există conexiuni directe între clienți. Presupunem că fiecare server poate ruta (trimite) mesaje către serverele cu care este direct conectat și că rețeaua este conexă (oricare două noduri pot comunica). Pentru a face rețeaua mai fiabilă sunt luate în considerare două scenarii:

- (a) Orice client să fie conectat direct la cel puțin 2 servere și orice pereche de servere aflate la distanța 2 să fie conectate direct (prin cabluri). Demonstrați că, în această nouă rețea, dacă unul dintre servere cade, clienții pot comunica cu toate serverele rămase.
- (b) Se adaugă cabluri de rețea astfel ca fiecare legătură directă între două servere, e , se află pe două circuite care se intersectează doar în e . Arătați că rețeaua rămâne conexă dacă două legături directe între servere cad.

(2 + 1 = 3 points)

3. Fie $G = (V, E)$ un graf conex. Un subgraf indus H al lui G este numit *b-subgraf* dacă este 2-conex și orice subgraf indus al lui G care conține pe H strict nu este 2-conex.

- (a) Arătați că dacă două b-subgrafuri ale lui G au noduri în comun, atunci intersecția lor este un punct de articulație al lui G și că orice punct de articulație este în intersecția a cel puțin două b-subgrafuri.
- (b) Se definește următorul meta-graf bipartit $\mathcal{G} = (\mathcal{V}_1, \mathcal{V}_2; \mathcal{E})$, unde $\mathcal{V}_1$ este mulțimea b-subgrafurilor lui G , $\mathcal{V}_2$ este mulțimea punctelor de articulație ale lui G , iar $H \in \mathcal{V}_1$ este adiacent cu $u \in \mathcal{V}_2$ în $\mathcal{G}$ dacă $u \in V(H)$. Demonstrați că $\mathcal{G}$ este un arbore.

Considerăm următorul algoritm:

for ($v \in V$) **do**

$order[v] \leftarrow -1$; $parent[v] \leftarrow -1$; $high[v] \leftarrow -1$; // *order, parent și high sunt variabile globale*
 fie x_0 un nod al lui G ; $k \leftarrow 0$; // *k este o variabilă globală*
 Traverse(x_0);

unde

Traverse(x)

$k++$; $order[x] \leftarrow k$; $high[x] \leftarrow order[x]$; $children \leftarrow 0$;

for ($v \in A(x)$) **do**

if ($order[v] = -1$) **then**

$parent[v] \leftarrow x$;

$children++$;

Traverse(v);

if ($parent[x] == -1$) and ($children == 2$) **then**

write("x is a cut-vertex");

if ($parent[x] \neq -1$) **then**

$high[x] \leftarrow \min\{high[v], high[x]\}$;

if ($order[x] \leq high[v]$) **then**

write("x is a cut-vertex");

else if ($v \neq parent[x]$) and ($order[x] > order[v]$) **then**

$high[x] \leftarrow \min\{high[x], order[v]\}$;

- (c) Arătați că algoritmul de mai sus este corect: demonstrați că un nod x este punct de articulație al grafului G dacă și numai dacă x este rădăcină și are cel puțin doi copii sau x nu este rădăcină și are un copil v astfel că $order[x] \leq high[v]$. (Rădăcina este nodul x_0 , pentru care $parent[x_0] = -1$.)

(2+1+2 = 5 points)

4. Fie $G = (V, E)$ un graf conex și $c : E \rightarrow \mathbb{R}$ o funcție de cost pe muchii așa încât muchii diferite au costuri diferite.

```

 $T \leftarrow (V, \emptyset);$ 
while ( $T$  este neconex) do
    fie  $T_1, T_2, \dots, T_p$  componentele conexe ale lui  $T$ ; //  $p \geq 2$ 
    for ( $i = \overline{1, p}$ ) do
        fie  $e_i \in E$  astfel încât  $c(e_i) = \min \{c(uv) : uv \in E, u \in V(T_i), v \notin V(T_i)\};$ 
         $E(T) \leftarrow E(T) \cup \{e_1, e_2, \dots, e_p\};$ 
return  $T$ ;

```

Demonstrați că

- (a) după fiecare iterație **while** numărul de componente conexe ale lui T se înjumătățește (cel puțin);
- (b) după fiecare iterație **while** T este aciclic;
- (c) T este un arbore parțial de cost minim al lui G ;
- (d) numărul de iterații **while** este $\mathcal{O}(\log n)$ și fiecare iterație **while** are complexitatea $\mathcal{O}(m)$, astfel complexitatea totală a algoritmului este $\mathcal{O}(m \log n)$. ($n = |V|, m = |E|$)

(1 + 1 + 1 + 1 = 4 points)